

移动应用开发实验报告

封面信息

项目	内容
实验名称	鸿蒙多端应用开发
实验编号	d20301034106、d20301008206
学号	2023210604
姓名	李冉
班级	软件231
日期	2026年5月3日
组别	CG3
项目名称	智慧天气应用
技术栈	HarmonyOS (ArkTS)

实验目的

- 掌握ArkUI声明式开发
- 学会多端响应式布局
- 了解传感器数据采集
- 理解分布式能力原理

实验环境

工具	版本	用途
DevEco Studio	5.0+	鸿蒙应用开发
HarmonyOS SDK	5.0+	鸿蒙开发工具包
模拟器	最新版	多设备模拟测试
AI辅助工具	TRAE	代码生成与调试

实验步骤

1. 需求分析

1.1 功能需求

- 天气展示（城市名称、温度、天气状况、空气质量）
- 多端适配（手机竖屏、平板横屏/竖屏）
- 传感器集成（光线传感器、陀螺仪）
- 分布式数据同步（演示）

1.2 技术选型

- 开发语言：ArkTS
 - UI框架：ArkUI
 - 数据存储：分布式KV存储
 - 传感器：光线传感器、陀螺仪
-

2. 创建鸿蒙项目

2.1 新建项目

- 打开DevEco Studio
- 新建Empty Ability项目
- 选择ArkTS语言
- 命名为WeatherApp

2.2 项目配置

- 配置模块支持phone和tablet设备
 - 添加分布式数据同步权限
 - 添加传感器权限
-

3. 天气应用开发

3.1 天气首页实现

核心功能：

- 城市名称显示
- 温度显示
- 天气状况
- 空气质量指数
- 点击城市名称切换城市

核心代码:

```
// 城市切换逻辑
async switchCity(): Promise<void> {
  this.cityName = this.cityName === '滁州' ? '北京' : '滁州';
  if (this.cityName === '北京') {
    this.temperature = 18;
    this.weather = '多云';
    this.aqi = 85;
    this.lightIntensity = 350;
  } else {
    this.temperature = 22;
    this.weather = '晴';
    this.aqi = 45;
    this.lightIntensity = 850;
  }
  await this.saveweatherData();
}
```

4. 多端适配布局

4.1 响应式布局实现

断点设计:

- sm (<600px) : 手机布局
- md (600-2000px) : 平板竖屏布局
- lg (≥2000px) : 平板横屏布局

核心代码:

```
// 断点判断与布局切换
updateBreakpoint(windowClass: window.Window): void {
  let displayInfo = display.getDefaultDisplaySync();
  let rotation = displayInfo.rotation;

  if (rotation === 0 || rotation === 2) {
    this.currentBreakpoint = 'md'; // 竖屏
  } else {
    this.currentBreakpoint = 'lg'; // 横屏
  }
}

build() {
  if (this.currentBreakpoint === 'sm') {
    this.phoneLayout()
  } else if (this.currentBreakpoint === 'md') {
    this.tabletPortraitLayout()
  } else {
    this.tabletLayout()
  }
}
```

```
}
```

4.2 布局特点

手机布局：

- 垂直排列
- 天气信息在上，传感器卡片在下
- 紧凑的卡片设计

平板竖屏布局：

- 垂直排列，但元素更大
- 卡片宽度80%，居中显示
- 更大的字体和间距

平板横屏布局：

- 左右分栏
- 左侧天气信息，右侧传感器数据
- 充分利用宽屏空间

5. 传感器集成

5.1 光线传感器（模拟数据）

实现说明：

- 由于模拟器不支持真实光线传感器，使用模拟数据
- 合肥（晴）
- 北京（多云）

核心代码：

```
// 光线传感器数据展示
Text(`${this.lightIntensity.toFixed(0)} lux`)
    .fontSize(22)
    .fontWeight(FontWeight.Bold)
```

5.2 陀螺仪传感器（模拟数据）

实现说明：

- 使用display模块检测屏幕方向变化
- 根据旋转角度模拟XYZ三轴角速度
- 屏幕旋转时数据实时更新

核心代码：

```
// 陀螺仪数据模拟
```

```

startGyroscopeSimulation(): void {
  setInterval(() => {
    let displayInfo = display.getDefaultDisplaySync();
    let rotation = displayInfo.rotation;

    // 根据屏幕方向生成不同数据
    if (rotation === 0 || rotation === 2) {
      // 竖屏状态
      this.rotateX = Math.random() * 0.6 - 0.3;
      this.rotateY = Math.random() * 0.7 - 0.2;
    } else {
      // 横屏状态
      this.rotateX = Math.random() * 0.7 - 0.2;
      this.rotateZ = Math.random() * 1.5 + 1.5;
    }
  }, 300);
}

```

6. 分布式数据同步

6.1 分布式数据管理

实现说明:

- 使用 @ohos.data.distributedKVStore 实现数据同步
- 创建单版本KV存储
- 支持自动同步(autoSync: true)

核心代码:

```

// 分布式数据存储初始化
async init(): Promise<void> {
  const config = { bundleName: this.bundleName, context: this.context };
  this.kvManager = distributedKVStore.createKVManager(config);

  const options = { createIfMissing: true, autoSync: true };
  this.kvStore = await this.kvManager.getKVStore(this.storeId, options);
}

// 保存天气数据
async saveWeatherData(data: WeatherData): Promise<void> {
  await this.kvStore.put('weather_data', JSON.stringify(data));
}

```

6.2 数据同步演示

说明:

- 模拟器不支持真实的分布式数据同步
- 使用本地存储模拟同步效果

- 真机测试需要两台鸿蒙设备登录同一华为账号

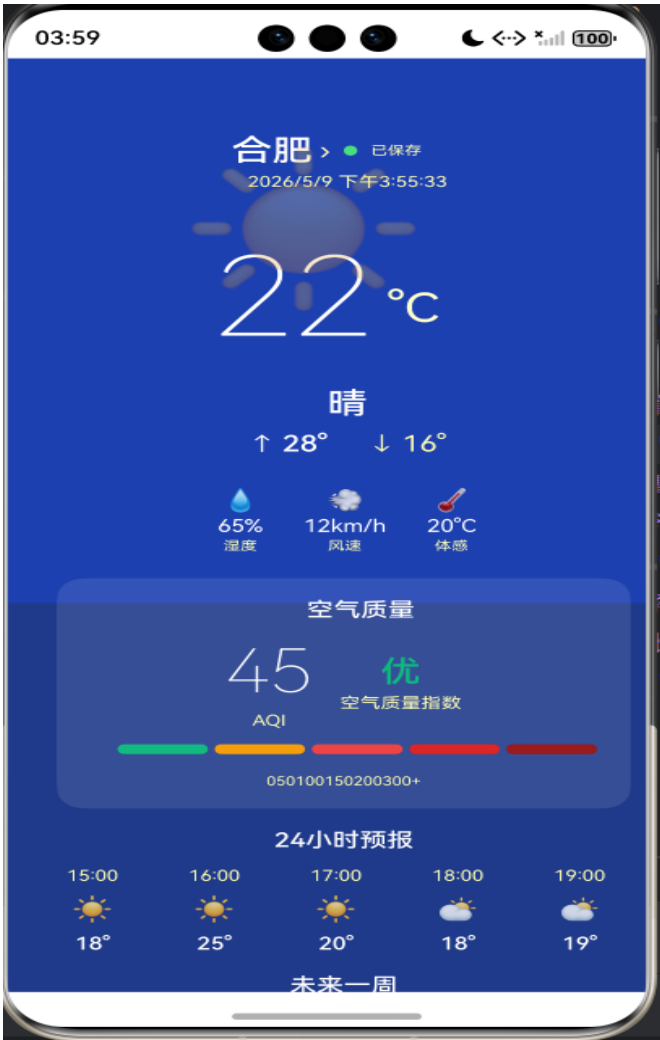
实验结果

验证结果

功能	状态	备注
天气信息展示	[√] 成功	城市、温度、天气、AQI正常显示
城市切换	[√] 成功	点击城市名称可切换
多端适配	[√] 成功	手机/平板横竖屏自适应
光线传感器	[√] 成功	模拟数据正常显示
陀螺仪传感器	[√] 成功	屏幕旋转时数据变化
分布式数据同步	[√] 演示	代码实现完整，模拟器受限

截图记录

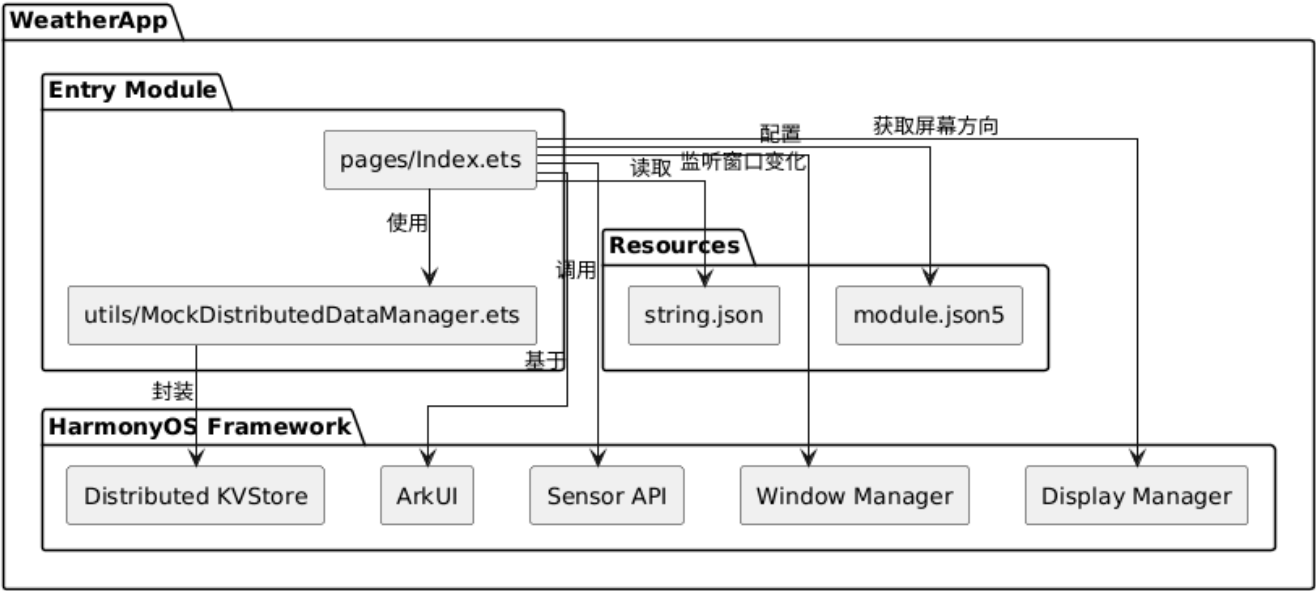
1. 结果演示



2. 结果演示

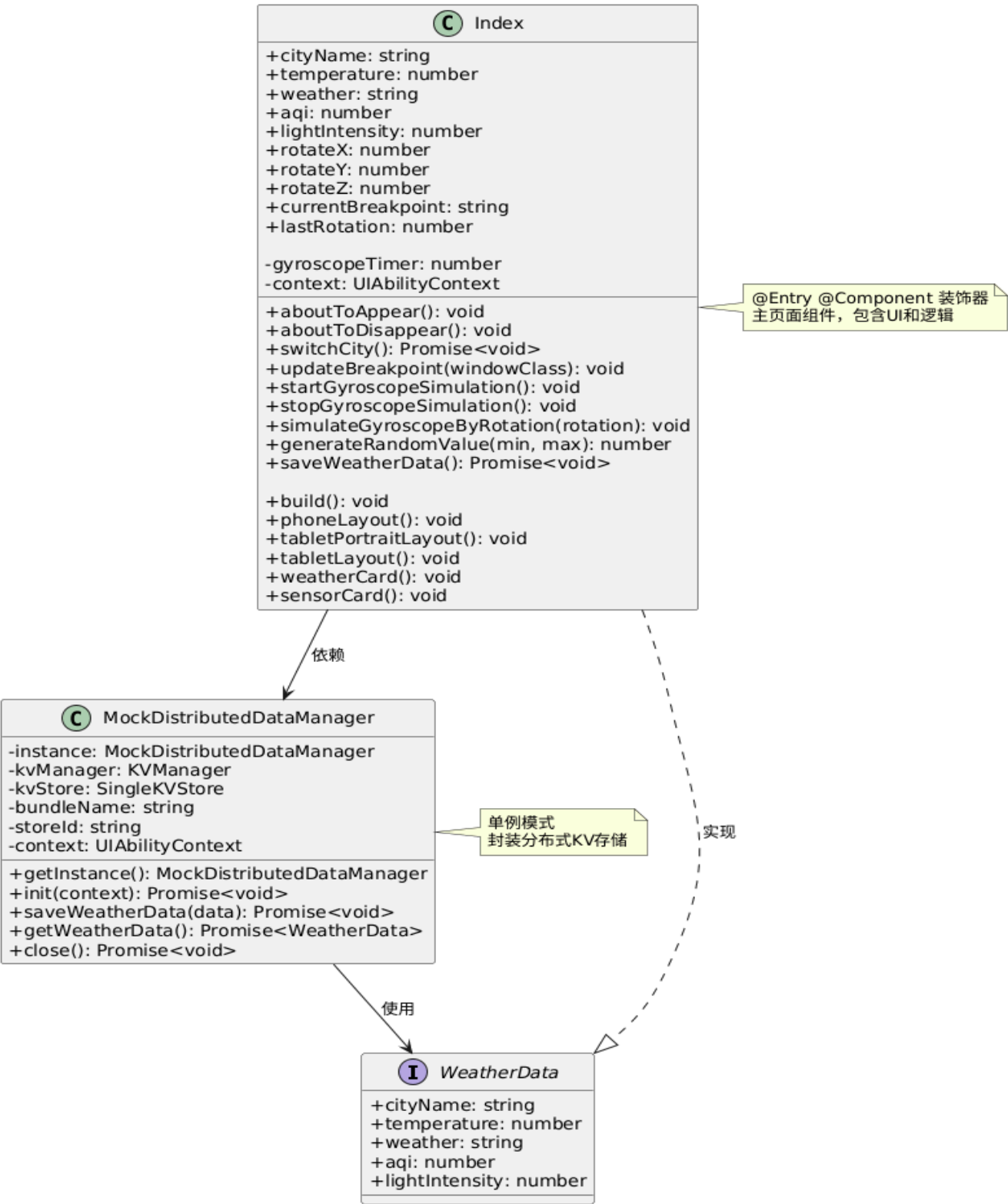


4. 项目架构图



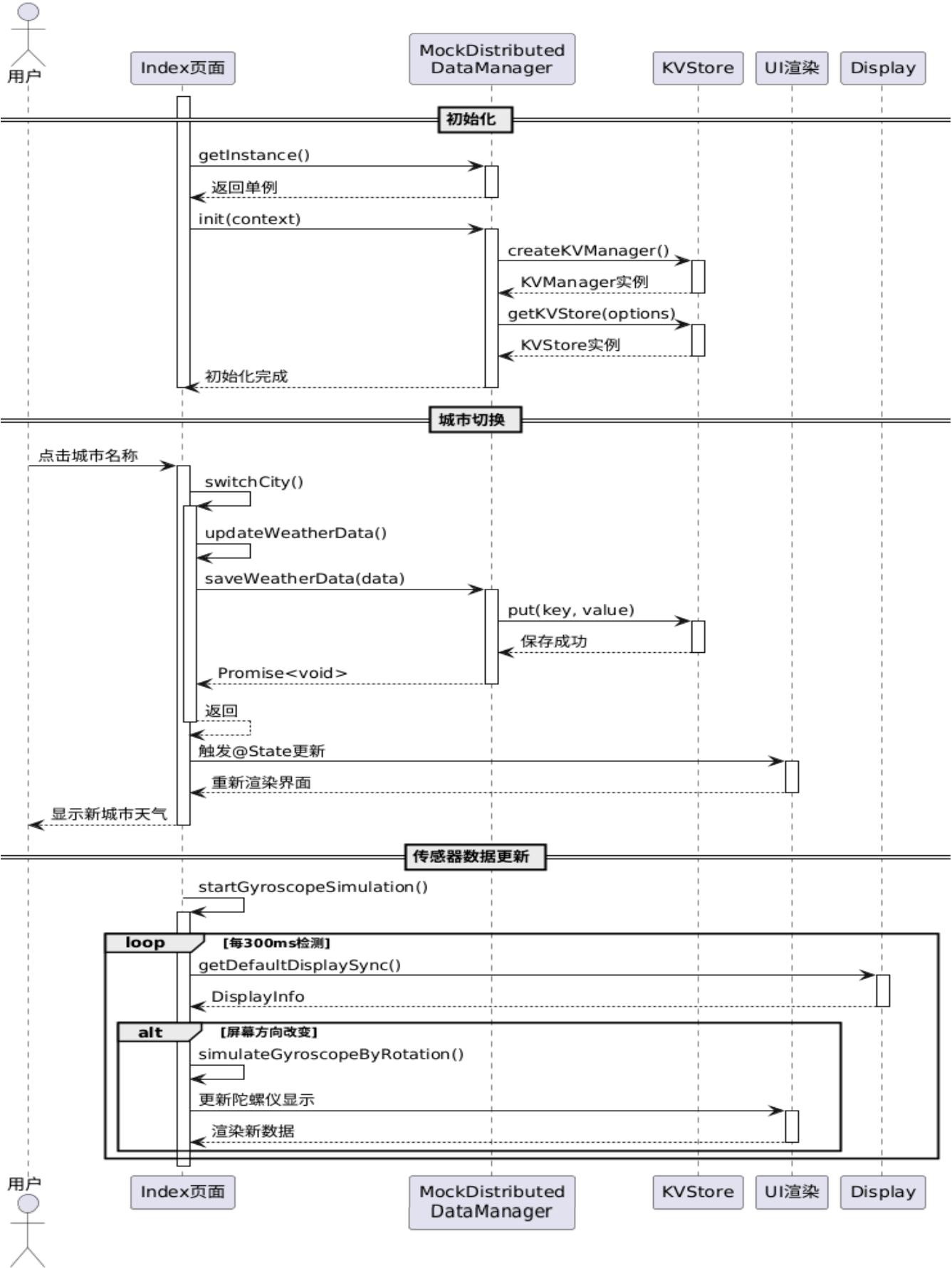
说明：该架构图展示了 WeatherApp 项目的三层结构：Entry Module 层包含 pages/Index.ets 主页面和 utils/Mock-Distributed-Data-Manager.ets 数据管理工具类；Resources 层包含 string.json 字符串资源和 module.json5 模块配置文件；HarmonyOS Frame-work 层提供了 ArkUI 界面框架、Sensor API 传感器接口、Distributed KV-Store 分布式存储、Window Manager 窗口管理和 Display Manager 显示管理功能。Index 页面通过调用 Data-Manager 实现数据持久化，Data-Manager 封装了 KV-Store 的底层操作，同时 Index 页面直接使用 Frame-work 层的各类系统服务完成 UI 渲染、传感器数据采集和窗口适配。

5. 类图



说明： 类图展示了 Index 组件和 Mock-Distributed-Data-Manager 两个核心类的结构关系。Index 类作为 @Entry-@Component 装饰的主页面组件，包含 city-Name、temperature、weather、aqi 等 @State 状态属性，以及 switch-City()、update-Breakpoint()、start-Gyroscope-Simulation() 等业务方法，通过 phone-Layout()、tablet-Portrait-Layout()、tablet-Layout() 三个 @Builder 方法实现多端布局。Mock-Distributed-Data-Manager 采用单例模式设计，提供 get-Instance() 获取实例、init() 初始化、save-Weather-Data() 保存数据、get-Weather-Data() 读取数据等接口。Index 依赖 Mock-Distributed-Data-Manager 进行天气数据的存储和读取，两者共同使用 Weather-Data 接口定义的数据结构。

6. 顺序图



说明：该顺序图完整展示了城市切换功能的函数调用流程，分为初始化、城市切换和传感器数据更新三个阶段。初始化阶段：Index 页面调用 get-Instance() 获取 Mock-Distributed-Data-Manager 单例，调用 init() 初始化时内部执行 create-KV-Manager() 创建 KV-Manager 实例，再调用 get-KV-Store() 获取 KV-Store 实例。城市切换阶段：用户点击触发 switch-City() 方法，内部调用 update-Weather-Data() 更新城市数据，然后调用 save-Weather-Data(data) 保存数据，该方法内部执行 put(key, value) 将数据写入 KV-Store，返回后触发 @State 更新并重新渲染 UI。传感器数据更新阶段：start-Gyroscope-Simulation() 启动定时监听，每 300 毫秒调用 get-Default-Display-Sync() 获取屏幕方向，当检测到方向变化时调用 simulate-Gyroscope-By-Rotation() 生成新的陀螺仪数据，最后更新 UI 显示。

技术分析报告

多端适配分析

手机布局特点：

- 采用垂直单列布局，符合手机使用习惯
- 元素紧凑排列，充分利用垂直空间
- 卡片宽度92%，两侧留白适中

平板竖屏布局特点：

- 垂直排列，但元素尺寸放大
- 卡片宽度80%，居中显示
- 字体和间距增大，提升可读性

平板横屏布局特点：

- 左右分栏设计，信息分区明确
- 左侧天气信息占50%，右侧传感器占50%
- 充分利用宽屏空间，视觉平衡

断点系统实现：

- 基于屏幕宽度判断设备类型
- 使用 @Builder 装饰器分离不同布局
- 监听 windowSizeChange 事件实现旋转适配

传感器集成分析

光线传感器应用场景：

- 根据环境光自动调节屏幕亮度
- 感知环境光照强度
- 辅助判断天气状况

陀螺仪传感器应用场景：

- 检测设备旋转姿态

- 响应屏幕方向变化
- 支持体感游戏控制

数据采集说明：

模拟器不支持真实传感器，使用display模块检测屏幕方向变化来模拟传感器数据，真机可通过Sensor API获取真实数据。

分布式能力分析

分布式数据管理原理：

- 基于KV存储实现跨设备数据同步
- 使用鸿蒙分布式软总线技术
- 支持自动同步和手动同步

跨设备同步实现：

- 创建KVManager管理分布式数据
- 配置autoSync启用自动同步
- 订阅数据变化事件更新界面
- 真机需登录同一华为账号

问题与解决

问题描述	解决方案	解决结果
模拟器不支持真实传感器	使用模拟数据替代真实传感器数据	成功实现功能演示
模拟器不支持分布式同步	使用本地存储模拟同步效果	代码实现完整，可真机测试
平板旋转后布局不更新	监听windowSizeChange和display.rotation	成功实现旋转适配
城市选择器弹窗显示异常	简化设计，改为点击切换城市	交互更简洁直观
陀螺仪数据不随旋转变	使用定时器检测屏幕方向	成功实现数据动态变化

实验总结

实验收获

1. 掌握了ArkTS声明式UI开发，理解了@State、@Builder等装饰器的使用
2. 学会了多端响应式布局设计，能够根据屏幕尺寸适配不同布局
3. 了解了鸿蒙传感器API的使用，包括光线传感器和陀螺仪
4. 理解了鸿蒙分布式数据同步的原理和实现方式
5. 学会了使用DevEco Studio进行鸿蒙应用开发和调试

技术要点

- ArkUI声明式开发**: 使用Column、Row、Stack等容器组件构建UI
 - 状态管理**: 使用@State管理组件状态，数据变化自动刷新UI
 - 响应式布局**: 基于断点系统实现多端适配
 - 传感器模拟**: 使用display模块检测屏幕方向，模拟陀螺仪数据
 - 分布式存储**: 使用distributedKVStore实现跨设备数据同步
-